

Aula 38: Dashboard Analítico - Teoria e Prática

Duração: 4 horas (2h teoria + 2h prática)

Parte Teórica (2h) - Visualização de Dados em Tempo Real

1. Por que Server-Sent Events (SSE)?

- **Conexão Unidirecional:** O servidor envia atualizações para o cliente sem necessidade de polling.
- **Protocolo HTTP/HTTPS:** Mais simples que WebSockets quando apenas o servidor precisa enviar dados.
- **Formato Padrão:** Dados em `text/event-stream` com eventos nomeados.

2. Diferenças entre SSE e WebSockets

Feature	SSE	WebSockets
Direção	Apenas servidor → cliente	Bidirecional
Protocolo	HTTP	WS (protocolo dedicado)
Reconexão	Automática	Manual
Casos de Uso	Dashboards, notificações	Chat, jogos, colaboração

3. Modelagem de Dados para o Dashboard

- **Estatísticas em Tempo Real:**

```
java

public record ChatStats(
    long totalMessages,
    long usersOnline,
    Map<String, Long> messagesPerUser, // Contagem por usuário
    Instant lastUpdate
) {}
```

4. Bibliotecas para Gráficos no Vue

- **Chart.js:** Leve e customizável.
- **Apache ECharts:** Para visualizações complexas.
- **Quasar QChart:** Integração nativa com o Quasar.

Parte Prática (2h) - Implementando o Dashboard

1. Backend - SSE com Spring WebFlux (1h)

Arquivo: StatsController.java

```
java

@RestController
@RequestMapping("/stats")
public class StatsController {

    @GetMapping(path = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<ChatStats> streamStats() {
        return Flux.interval(Duration.ofSeconds(2)) // Atualiza a cada 2s
            .flatMap(tick -> statsService.getRealTimeStats());
    }
}
```

Serviço de Estatísticas:

```
java

@Service
public class StatsService {

    public Mono<ChatStats> getRealTimeStats() {
        return Mono.zip(
            messageRepository.count(),
            userSessionRepository.countActiveSessions(),
            messageRepository.countByUser()
        ).map(tuple -> new ChatStats(
            tuple.getT1(),
            tuple.getT2(),
            tuple.getT3(),
            Instant.now()
        ));
    }
}
```

2. Frontend - Consumindo SSE e Exibindo Gráficos (1h)

Passo 1: Composable para SSE

Arquivo: src/composables/useSSE.ts

typescript

```
import { ref } from 'vue';

export function useSSE(url: string) {
  const data = ref<any>(null);
  const error = ref<Error | null>(null);

  const eventSource = new EventSource(url);

  eventSource.onmessage = (event) => {
    data.value = JSON.parse(event.data);
  };

  eventSource.onerror = (err) => {
    error.value = err;
  };

  return { data, error };
}
```

Passo 2: Componente do Dashboard

Arquivo: src/components/DashboardStats.vue

vue

```
<template>
  <div class="dashboard">
    <q-card>
      <q-card-section>
        <h5>Estatísticas do Chat</h5>
        <div v-if="stats">
          <p>Total de mensagens: {{ stats.totalMessages }}</p>
          <p>Usuários online: {{ stats.usersOnline }}</p>
          <q-chart
            type="bar"
            :data="chartData"
            :options="chartOptions"
          />
        </div>
      </q-card-section>
    </q-card>
  </div>
</template>
```

```

</div>
</template>

<script setup lang="ts">
import { useSSE } from '@composables/useSSE';
import { computed } from 'vue';

const { data: stats } = useSSE('http://localhost:8080/stats/stream');

const chartData = computed(() => ({
  labels: Object.keys(stats.value?.messagesPerUser || {}),
  datasets: [{
    label: 'Mensagens por usuário',
    data: Object.values(stats.value?.messagesPerUser || {}),
    backgroundColor: '#1976D2'
  }]
}));

const chartOptions = {
  responsive: true
};
</script>

```

Atividade Prática

Desafio: Adicione estas funcionalidades:

1. Filtro de Tempo:

- Adicione um seletor para filtrar estatísticas (últimos 5min, 1h, 24h).
- Modifique o endpoint para aceitar um parâmetro `?duration=PT5M`.

2. Gráfico de Linha para Histórico:

- Use `setInterval` para armazenar os últimos 10 pontos de dados.
- Exiba a evolução de `usersOnline` ao longo do tempo.

Solução Sugerida:

```

typescript

// No componente DashboardStats.vue
const timeRange = ref('PT5M');
const historicalData = ref<number[]>([]);

watch(stats, (newStats) => {
  if (newStats?.usersOnline) {

```

```
historicalData.value.push(newStats.usersOnline);
if (historicalData.value.length > 10) {
  historicalData.value.shift();
}
}
});

// Novo gráfico
const historyChartData = computed(() => ({
  labels: historicalData.value.map((_, i) => `Ponto ${i + 1}`),
  datasets: [{
    label: 'Usuários online (histórico)',
    data: historicalData.value,
    borderColor: '#FF6B6B'
  }]
}));
```

Recursos Adicionais

- Documentação SSE: [MDN EventSource](#)
- Chart.js + Vue: [vue-chartjs](#)

Próxima Aula: Deploy com Docker e otimizações finais! 🚀